

Certificats pour les preuves d'équivalence de circuits avec Isabelle/HOL

Alexis Beaucourt

29 mai 2026

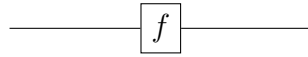
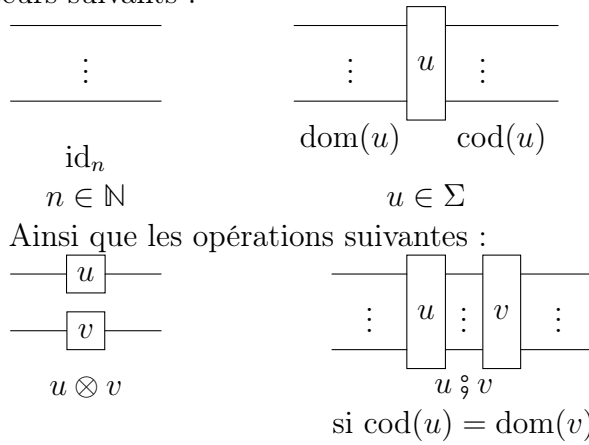


FIGURE 1 – My graph.

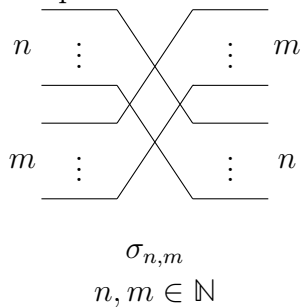
1 Définition des PRO/PROP

Chaque circuit u possède un domaine $\text{dom}(u) \in \mathbb{N}$ et un codomaine $\text{cod}(u) \in \mathbb{N}$

On définit les circuits de $\mathbf{PRO}_{\Sigma, \mathcal{E}}$ comme étant les circuits formés à partir des générateurs suivants :



Cependant, on quotiente le tout par les égalités de $\mathcal{E}$, en plus des égalités suivantes :
[insérer image]
On peut ensuite définir les $\mathbf{PROP}_{\Sigma, \mathcal{E}}$ comme étant des $\mathbf{PRO}_{\Sigma \cup \Sigma_0, \mathcal{E} \cup \mathcal{E}_0}$, où Σ_0 contient :



Et les $\mathcal{E}_0$ sont les égalités :

[insérer image]

Pour le restant de ce rapport, on notera $\simeq$ la relation d'équivalence entre deux termes.

2 simp dans Isabelle

2.1 en général

En Isabelle/HOL la tactique `simp` permet de modifier (ou prouver) un but en appliquant des règles de réécriture. En particulier, `simp` peut être invoquée sous plusieurs formes :

- `simp` utilise un ensemble de règles de réécriture par défaut. Cet ensemble est un mix de règles judicieusement choisies par les créateurs de bibliothèques et de règles automatiquement générées par des mots clés comme `function`, `fun`, `inductive`, etc.
- `(simp add: regle1 regle2)` permet à l'utilisateur de rajouter ses propres règles de réécriture dans l'ensemble utilisé par `simp`
- `(simp only: regle1 regle2)` permet à l'utilisateur de ne pas utiliser les règles par défaut, mais seulement celles qu'il choisit
- `(simp cong: regle1 regle2)` permet à l'utilisateur de rajouter des règles de congruence

Les options `add`, `only` et `cong` peuvent être utilisées dans la même invocation de `simp`, par exemple `(simp only: regle1 cong: regle2)` n'utilise que 2 règles, dont une de congruence.

Une règle de réécriture est un lemme (qui doit être démontré dans Isabelle) sous la forme $P_1 \implies \dots \implies P_n \implies A = B$. Il représente la règle $A \rightarrow B$ si $P_1, \dots, P_n$

En pratique, le simplificateur va essayer de prouver les conditions $P_1, \dots, P_n$ en s'appuyant récursivement sur chacune des conditions, et réécrire A en B une fois qu'il a réussi. S'il n'arrive pas à prouver une des conditions (parce qu'il n'a pas assez de règles ou bien parce que la condition est fausse), alors il abandonne l'utilisation de cette règle et en essaye d'autres.

Une règle de congruence est un lemme de la forme $P_1 \implies \dots \implies P_n \implies f(x) = f(y)$. Elle s'applique avant les règles de réécriture, dès que le simplificateur reconnaît que les deux côtés de l'égalité commencent par f . De plus, les règles de congruence se distinguent des règles de réécriture par le fait qu'il ne s'agit pas ici de remplacer $f(x)$ par $f(y)$, mais de prouver $f(x) = f(y)$.

Par exemple, si f est une fonction paire, on peut avoir la règle de congruence $x = -y \implies f(x) = f(y)$, ce qui permet au simplificateur de démontrer $f(-3) = f(3)$

Si le simplificateur arrive à un terme sur lequel il n'y a pas de règles applicables (ou les seules règles potentiellement applicables ont une condition dont la preuve a échoué), alors il s'arrête. Si de plus il s'arrête sur un terme de la forme $\mathbf{t} = \mathbf{t}$, alors il considère son but comme démontré.

Pour la certification d'équivalence de circuits, il est judicieux d'utiliser `simp`, car notre forme normale est calculée à l'aide de règles de réécriture. Cependant, les règles de réécriture indiquées ci-dessus ne sont pas suffisantes pour le bon fonctionnement de `simp`. En effet, il lui faut aussi :

- des règles pour évaluer certaines fonctions (`dom`, `cod`, ...)
- des règles pour faire de l'arithmétique sur les entiers naturels
- des règles pour traiter des inégalités sur les entiers naturels
- des règles pour évaluer des opérations booléennes

Il est aussi nécessaire d'écrire les règles d'une manière appropriée pour `simp` : une règle comme $f(n + 1) = g(n)$ ne s'applique pas sur $f(13)$, mais sur $f(12 + 1)$. Il convient de prendre à la place une règle comme $n \geq 1 \implies f(n) = g(n - 1)$

Il est également important de prendre en compte la difficulté posée par la soustraction sur les entiers naturels. En effet, si $n \geq m$, $m - n$ est défini comme étant 0. Par exemple, $1 - 2 = 0$ peut être démontré en Isabelle. Il faut donc exprimer $n - m$ par $\text{nat } (\text{int } n - \text{int } m)$ (on passe par les entiers relatifs)

2.2 avec les PRO

2.3 avec les Perm

3 benchmark

3.1 Méthodologie

3.2 PRO bench

3.3 Perm bench

4 PROP

Afin d'espérer pouvoir obtenir un résultat similaire sur les $\mathbf{PROP}_{\Sigma, \emptyset}$, on veut définir une forme normale.

4.1 Numérotation

Pour aider à définir une forme normale, on va ajouter des informations à notre circuit. Cela permet de donner des informations globales sur le circuit de manière locale.

Remarque 4.1.1. *On peut voir le circuit comme un graphe (plus précisément un DAG)*

Théorème 4.1.2. $t_1 \simeq t_2 \iff G(t_1) = G(t_2)$

Corollaire 4.1.3. *Toute quantité qui ne dépend que de $G(t)$ est invariante pour toute la classe d'équivalence de t*

Définition 4.1.4. *On définit la profondeur $\rho(u)$ d'un générateur $u \in \Sigma$ dans un circuit comme étant le nombre maximal de générateur traversés vers la gauche.*

Autrement dit, si les générateurs précédant v sont $u_1, \dots, u_n$, alors

$$\rho(v) = \begin{cases} 0 & \text{si } n = 0 \\ 1 + \max_{1 \leq i \leq n} \rho(u_i) & \text{sinon} \end{cases}$$

Lemme 4.1.5. *Comme la notion de "précéder" ne dépend que du graphe sous-jacent, la profondeur ainsi définie est indépendante de l'élément de la classe d'équivalence choisie.*

Définition 4.1.6. *On note la "vraie profondeur" $p(v)$ d'une porte (c'est-à-dire un générateur ou bien un swap) par l'équation suivante :*

$$p(v) = \begin{cases} 2 \times \rho(v) + 1 & \text{si } v \text{ est un générateur} \\ 2 \times \max_{1 \leq i \leq n} \rho(u_i) & \text{si } v \text{ est un swap et } (u_i)_i \text{ les générateurs le précédant} \end{cases}$$

Définition 4.1.7. On définit la priorité d'un fil en faisant un parcours en profondeur sur le graphe. On obtient ainsi un ordre topologique sur les fils.

Lemme 4.1.8. Comme cet ordre ne dépend que du graphe sous-jacent, il est équivalent pour tout élément de la classe d'équivalence.

Lemme 4.1.9. On peut donc munir un circuit des profondeurs et des priorités sans toutefois modifier les classes d'équivalence.

Définition 4.1.10. On dit d'un circuit (annoté) est en forme normale si et seulement si :

- il est "découpé en couches" selon la profondeur p
- au sein de chaque couche, les portes sont organisées selon la profondeur de leurs fils de sortie

Lemme 4.1.11. La forme normale existe, et est unique.

Démonstration. Existence par correction (DFS) et construction

Unicité par ordre total des priorités

□

Définition 4.1.12. On note NF la procédure qui transforme t en forme normale.

Propriété 4.1.13. $NF(NF(t)) = NF(t)$

Lemme 4.1.14. $t_1 \simeq t_2 \iff NF(t_1) = NF(t_2)$

Démonstration. NF ne dépend que du graphe sous-jacent et est unique.

□